

Analyse de données de la toxicité du paracétamol sur le foie

Rabah ACHOUR, Mohand Arezki BRANECHI

A) Motivation

On souhaite étudier l'influence du paracétamol sur la toxicité hépatique à partir d'un jeu de données transcriptomiques sur des rats. On dispose de données d'expression génique (3116 gènes) mesurées sur 64 rats exposés à différentes doses de paracétamol. L'objectif est d'identifier les gènes dont l'expression est significativement affectée par la dose de paracétamol.

```
rm(list=ls())
library(MultiVarSel)

## Le chargement a nécessité le package : Matrix

## Le chargement a nécessité le package : glmnet

## Loaded glmnet 4.1-10

## Le chargement a nécessité le package : ggplot2

library(ggplot2)
```

B) Exploration des données

1) Examen brut des données

```
load("liver_data.Rdata")
ls()
```

```
## [1] "liver.toxicity"
```

On a un objet `liver.toxicity`. Regardons la structure globale de l'objet

```
summary(liver.toxicity)
```

```
##           Length Class      Mode
## gene      3116   data.frame list
## clinic     10   data.frame list
## treatment   4   data.frame list
## gene.ID     2   data.frame list
```

On a 4 éléments dans l'objet `liver.toxicity`. Regardons chacun d'eux.

`treatment`

```
table1 = liver.toxicity$treatment
str(table1)
```

```
## 'data.frame':   64 obs. of  4 variables:
## $ Animal.Number : int  202 203 204 206 208 209 210 211 212 213 ...
## $ Treatment.Group: Factor w/ 16 levels "1500MG/K18 HR",...: 16 16 13 13 14 14 15 15 15 16 ...
## $ Dose.Group     : num  50 50 50 50 50 50 50 50 50 50 ...
## $ Time.Group     : int   6 6 18 18 24 24 48 48 48 6 ...
```

```
table1[1:3, ]
```

```
##      Animal.Number Treatment.Group Dose.Group Time.Group
## 202           202      50 MG/KG6 HR          50          6
## 203           203      50 MG/KG6 HR          50          6
## 204           204      50 MG/KG18 HR          50         18
```

La colonne `Treatment.Group` contient la même information que `Dose.Group`, enlevons-la.

```
table1$Treatment.Group <- NULL
table1$Time.Group <- NULL
table1[1:3, ]
```

```
##      Animal.Number Dose.Group
## 202           202          50
## 203           203          50
## 204           204          50
```

Examinons la répartition du facteur `Dose.Group`.

```
table(table1$Dose.Group)
```

```
##
##  50  150 1500 2000
##  16   16   16   16
```

La distribution des échantillons est parfaitement équilibrée.
Chaque niveau de *Dose.Group* (50, 150, 1500, 2000) contient 16 échantillons.

`gene`

```
table2 = liver.toxicity$gene
dim(table2)
```

```
## [1] 64 3116
```

```
table2[1:3, 1:5]
```

```
##      A_43_P14555 A_43_P22290 A_43_P20792 A_43_P21286 A_43_P12995
## ID202      0.05169      -0.08120      0.00617      0.00003      0.00676
## ID203      0.01548      0.17515      0.04845      0.04730      0.08869
## ID204     -0.01509     -0.04790     -0.02313     -0.02269     -0.07799
```

```
clinic
```

```
table3 = liver.toxicity$clinic
str(table3)
```

```
## 'data.frame': 64 obs. of 10 variables:
## $ BUN.mg.dL. : num 12 15 17 19 20 17 16 14 12 11 ...
## $ Creat.mg.dL. : num 0.6 0.7 0.8 0.7 0.7 0.7 0.7 0.6 0.7 0.7 ...
## $ TP.g.dL. : num 7.3 7.5 7.6 7.5 7.8 7.7 7.6 7.4 7.5 7.3 ...
## $ ALB.g.dL. : num 4.9 5 5 5 5.1 5.1 5.1 4.9 5.1 4.9 ...
## $ ALT.IU.L. : int 73 84 54 369 59 57 67 57 54 97 ...
## $ SDH.IU.L. : int 23 23 14 81 16 10 21 19 20 26 ...
## $ AST.IU.L. : int 87 110 129 450 75 80 93 90 76 135 ...
## $ ALP.IU.L. : int 320 325 381 401 359 335 324 309 317 251 ...
## $ TBA.umol.L. : int 9 8 4 13 3 13 8 11 6 7 ...
## $ Cholesterol.mg.dL.: int 85 87 84 82 85 88 82 92 86 77 ...
```

```
table3[1:3, ]
```

```
##      BUN.mg.dL. Creat.mg.dL. TP.g.dL. ALB.g.dL. ALT.IU.L. SDH.IU.L. AST.IU.L.
## 202          12          0.6       7.3         4.9         73          23          87
## 203          15          0.7       7.5         5.0         84          23         110
## 204          17          0.8       7.6         5.0         54          14         129
##      ALP.IU.L. TBA.umol.L. Cholesterol.mg.dL.
## 202         320          9                85
## 203         325          8                87
## 204         381          4                84
```

```
gene.ID
```

```
table4 = liver.toxicity$gene.ID
str(table4)
```

```
## 'data.frame': 3116 obs. of 2 variables:
## $ geneBank : Factor w/ 2397 levels "", "AA800287", ...: 1 1 1315 434 1901 106 1665 1773 1 1193 ...
## $ gene.title: Factor w/ 2079 levels "", "1-acylglycerol-3-phosphate 0-acyltransferase 5 (lysophosphatidylcholine acyltransferase)", ...: 1 1 1315 434 1901 106 1665 1773 1 1193 ...
```

```
table4[1:3, ]
```

```
##                geneBank                gene.title
## A_43_P14555
## A_43_P22290
## A_43_P20792 NM_001109576 tRNA splicing endonuclease 54 homolog (S. cerevisiae)
```

On a donc 4 dataframes dans l'objet `liver.toxicity`

- **gene** : données d'expression génique
 $\mathbf{G} \in \mathbb{R}^{64 \times 3116}$
64 échantillons $\times$ 3116 gènes (valeurs numériques).
- **clinic** : variables cliniques
 $\mathbf{C} \in \mathbb{R}^{64 \times 10}$
64 échantillons $\times$ 10 mesures cliniques (numériques et entières).
- **treatment** : groupes de traitement
 $\mathbf{T} \in \mathbb{R}^{64 \times 2}$
64 échantillons $\times$ 3 descripteurs (numéro d'animal, dose [numérique], temps [entier]).
- **gene.ID** : identifiants des gènes
 $\mathbf{I} \in \mathbb{R}^{3116 \times 2}$
3116 gènes $\times$ 2 champs d'annotation (geneBank, gene.title en facteurs).

2) Valeurs manquantes

```
which(is.na(table1))
```

```
## integer(0)
```

```
which(is.na(table2))
```

```
## integer(0)
```

```
which(is.na(table3))
```

```
## integer(0)
```

On n'a pas de valeurs manquantes. Concentrons nous sur les données **gene** et **treatment**.

3) Visualisation des données de gènes

Les données d'expression génique sont de haute dimension. Pour une visualisation de la variabilité, on peut utiliser une analyse en composantes principales (ACP).

```
pca_samples <- prcomp(table2, scale. = TRUE)
```

Regardons la variance expliquée par chaque composante principale.

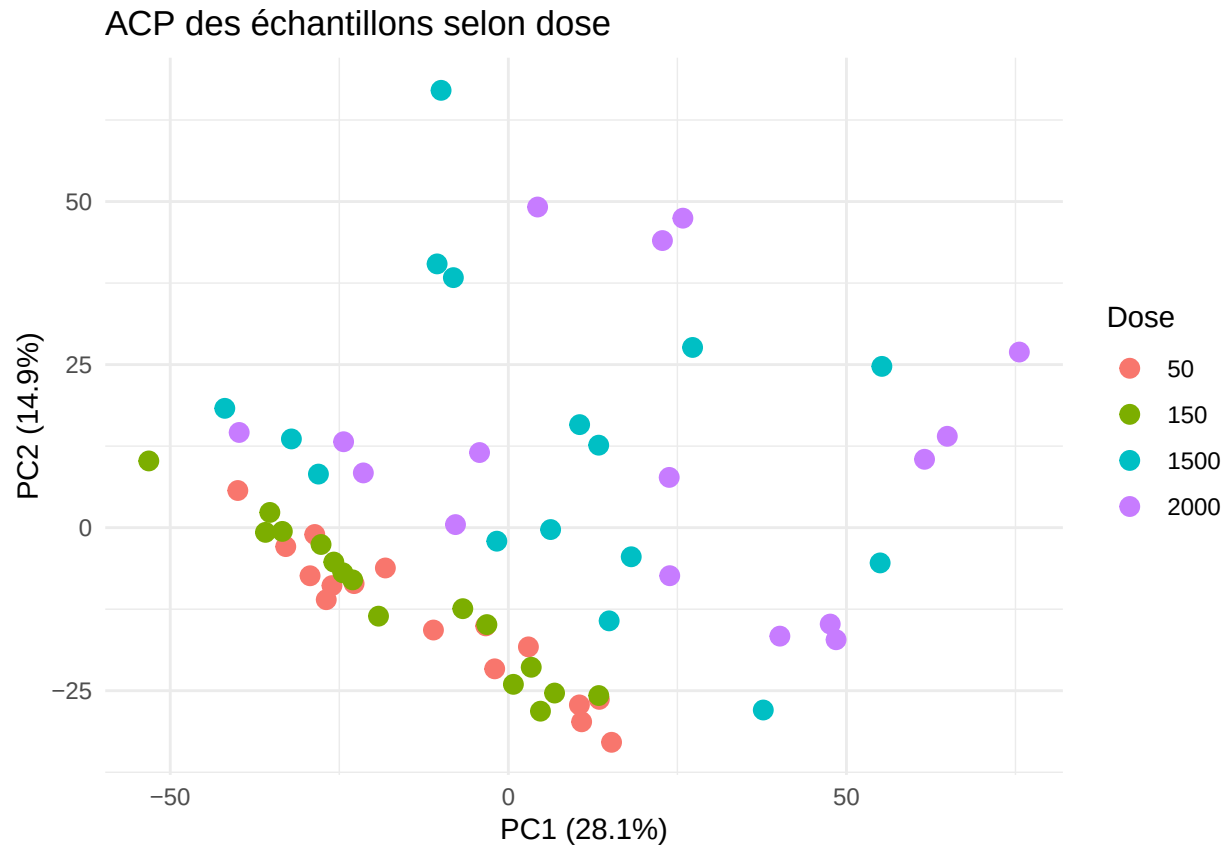
```
var_explained <- pca_samples$sdev^2 / sum(pca_samples$sdev^2)
round(var_explained[1:10], 3)
```

```
## [1] 0.281 0.149 0.080 0.054 0.046 0.039 0.035 0.025 0.024 0.018
```

Les deux premières composantes principales expliquent une part significative de la variance totale : PC1 capture 28.1 % et PC2 14.9 %, soit 43 % à elles deux. Cela indique que l'expression génique présente une structure non aléatoire.

Projection des échantillons avec coloration et formes par facteurs Dose et Périodicité

```
df_pca <- data.frame(PC1 = pca_samples$x[,1],
                     PC2 = pca_samples$x[,2],
                     Dose = factor(table1$Dose.Group))
ggplot(df_pca, aes(x = PC1, y = PC2, color = Dose)) +
  geom_point(size = 3) +
  labs(title = "ACP des échantillons selon dose",
       x = paste0("PC1 (", round(var_explained[1]*100,1), "%)"),
       y = paste0("PC2 (", round(var_explained[2]*100,1), "%)")) +
  theme_minimal()
```



On observe une séparation partielle des échantillons selon la dose, suggérant que ce facteur influence l'expression génique. Cette ACP justifie une analyse plus approfondie pour quantifier cet effet.

4) Réduction de la dimensionnalité

Pour alléger les calculs de la sélection de variables, on peut réduire la dimensionnalité en sélectionnant les gènes les plus variables. Le choix de 500 gènes vient d'un calcul empirique basé sur la RAM disponible (15 Go).

```
gene_importance <- rowSums(pca_samples$rotation[, 1:10]^2)
k <- 500
selected_genes <- order(gene_importance, decreasing = TRUE)[1:k]
table2_reduced <- table2[, selected_genes]
dim(table2_reduced)
```

```
## [1] 64 500
```

C) Modélisation linéaire multivariée à un facteur

1) Intuition de la modélisation

Le paracétamol est connu pour provoquer une toxicité hépatique à forte dose. Dans notre jeu de données, nous avons un facteur expérimental :

- **Dose.Group** : la quantité de paracétamol administrée aux rats.

Notre hypothèse est que :

- L'expression de certains gènes hépatiques **varie en fonction de la dose** administrée.

Cette intuition justifie l'utilisation, d'un **modèle linéaire multivarié** pour tester :

- l'effet principal de la dose.

2) Construction de la matrice de design X

Pour ajuster un modèle linéaire multivarié, on construit la matrice de design $X \in \mathbb{R}^{64 \times p}$ qui encode le facteur expérimental *Dose.Group*.

Le modèle multivarié s'écrit :

$$Y = X\beta + \varepsilon$$

où :

- $Y \in \mathbb{R}^{64 \times 1500}$ contient les expressions des 1500 gènes,
- X contient les colonnes indicatrices (contrastes) associées à la dose,
- $\beta \in \mathbb{R}^{p \times 1500}$ est la matrice des coefficients estimés pour tous les gènes simultanément,
- ε est la matrice des résidus.

Dans notre cas, la matrice X contient :

- une colonne d'intercept,
- trois colonnes pour les contrastes de dose (4 niveaux $\rightarrow$ 3 degrés de liberté),

Ainsi,

$$p = 1 + 3 = 4.$$

donc

$$X \in \mathbb{R}^{64 \times 4}.$$

```
table_XY <- data.frame(table1[-1], table2_reduced)
table_XY$Dose.Group <- factor(table_XY$Dose.Group)
table_XY[1:3, 1:10]
```

```
##      Dose.Group A_43_P17008 A_43_P19369 A_43_P17618 A_42_P726191 A_43_P19193
## 202          50    -0.00184    0.03701    0.01749    0.04866    0.00361
## 203          50     0.06408    0.04200    0.07975    0.04893    0.04925
## 204          50    -0.00740   -0.00032   -0.00690    0.09739   -0.03760
##      A_42_P815757 A_43_P19591 A_43_P10900 A_43_P18416
## 202     0.08862    -0.02615   -0.00965    0.01797
## 203     0.54198     0.11964    0.08473    0.03371
## 204     0.02901     0.06576   -0.00089   -0.00791
```

```
dim(table_XY)
```

```
## [1] 64 501
```

Matrice Y et X

Modélisation par un modèle linéaire multivarié

```
Y <- as.matrix(table2_reduced)
Y[1:3, 1:5]
```

```
##      A_43_P17008 A_43_P19369 A_43_P17618 A_42_P726191 A_43_P19193
## ID202    -0.00184    0.03701    0.01749    0.04866    0.00361
## ID203     0.06408    0.04200    0.07975    0.04893    0.04925
## ID204    -0.00740   -0.00032   -0.00690    0.09739   -0.03760
```

```
X <- model.matrix(lm(Y ~ Dose.Group + 0, data = table_XY))
X
```

```
##      Dose.Group50 Dose.Group150 Dose.Group1500 Dose.Group2000
## 202             1             0             0             0
## 203             1             0             0             0
## 204             1             0             0             0
## 206             1             0             0             0
## 208             1             0             0             0
## 209             1             0             0             0
## 210             1             0             0             0
## 211             1             0             0             0
## 212             1             0             0             0
## 213             1             0             0             0
## 214             1             0             0             0
## 216             1             0             0             0
## 217             1             0             0             0
## 220             1             0             0             0
## 221             1             0             0             0
## 223             1             0             0             0
## 302             0             1             0             0
## 303             0             1             0             0
## 306             0             1             0             0
## 307             0             1             0             0
## 308             0             1             0             0
## 310             0             1             0             0
```

```

## 311      0      1      0      0
## 312      0      1      0      0
## 314      0      1      0      0
## 315      0      1      0      0
## 316      0      1      0      0
## 317      0      1      0      0
## 318      0      1      0      0
## 319      0      1      0      0
## 320      0      1      0      0
## 324      0      1      0      0
## 402      0      0      1      0
## 403      0      0      1      0
## 404      0      0      1      0
## 405      0      0      1      0
## 406      0      0      1      0
## 407      0      0      1      0
## 411      0      0      1      0
## 412      0      0      1      0
## 413      0      0      1      0
## 414      0      0      1      0
## 416      0      0      1      0
## 419      0      0      1      0
## 420      0      0      1      0
## 421      0      0      1      0
## 423      0      0      1      0
## 424      0      0      1      0
## 501      0      0      0      1
## 503      0      0      0      1
## 505      0      0      0      1
## 506      0      0      0      1
## 508      0      0      0      1
## 509      0      0      0      1
## 510      0      0      0      1
## 512      0      0      0      1
## 513      0      0      0      1
## 514      0      0      0      1
## 516      0      0      0      1
## 518      0      0      0      1
## 520      0      0      0      1
## 521      0      0      0      1
## 522      0      0      0      1
## 524      0      0      0      1

```

```

## attr("assign")
## [1] 1 1 1 1
## attr("contrasts")
## attr("contrasts")$Dose.Group
## [1] "contr.treatment"

```

```
dim(X)
```

```
## [1] 64 4
```

```
colnames(X)
```

```
## [1] "Dose.Group50" "Dose.Group150" "Dose.Group1500" "Dose.Group2000"
```

```
Y <- scale(Y)
```

```
Y[1:3, 1:5]
```

```
##      A_43_P17008 A_43_P19369 A_43_P17618 A_42_P726191 A_43_P19193
## ID202  0.1954138  0.8226673  0.1616244  0.2754313  0.2470066
## ID203  1.0662297  0.9497040  1.4648235  0.2782059  0.6740362
## ID204  0.1219652 -0.1276890 -0.3488965  0.7761968 -0.1385738
```

On a bien les coefficients pour chaque gène (1500) et chaque paramètre du modèle

3) Analyse des résidus du modèle

Pour vérifier la validité du modèle multivarié et faire une sélection stable de gènes , examinons les résidus et leurs propriétés

- 1) **Hypothèse d'absence d'autocorrélation** : Les résidus doivent être indépendants entre eux, équivalents à un bruit blanc..

```
# residus = fit$residuals
residus = lm(as.matrix(Y) ~ X - 1)$residuals
dim(residus)
```

```
## [1] 64 500
```

```
pvalue = whitening_test(residus)
print(pvalue)
```

```
## [1] 4.851883e-06
```

```
dim(cov(residus))
```

```
## [1] 500 500
```

La p-valeur du test de blanchiment est très faible. Cela indique une extrême corrélation des résidus et que l'hypothèse d'absence d'autocorrélation est rejetée.

D) Modélisation de la dépendance des résidus et sélection de variables

4) Modélisation de la dépendance des résidus par estimation de la matrice de covariance Σ

Il existe plusieurs façons de modéliser la dépendance entre les résidus. Ces approches se distinguent par la manière dont elles estiment la matrice de covariance des erreurs :

- **Modèle AR(1)** : cas particulier du modèle autorégressif où chaque résidu dépend uniquement du précédent (structure très simple et entièrement paramétrique).
- **Modèle ARMA** : suppose que les résidus suivent un processus autorégressif à moyenne mobile (structure paramétrique flexible mais imposée).
- **Modèle non paramétrique** : aucune contrainte paramétrique n'est imposée ; la matrice de covariance est estimée directement et peut capturer des dépendances plus complexes

On va comparer ces trois approches pour choisir la plus adaptée aux données.

```
res_choice <- whitening_choice(residus, typeDeps = c("AR1", "nonparam", "ARMA"), pAR = 1, qMA = 1)
```

[illegible]

```
## convergence possible : optim renvoie un code = 1
## Warning in arima(residuals[i, ], order = c(pAR, 0, qMA)): problème de
## convergence possible : optim renvoie un code = 1
## Warning in arima(residuals[i, ], order = c(pAR, 0, qMA)): problème de
## convergence possible : optim renvoie un code = 1
## Warning in arima(residuals[i, ], order = c(pAR, 0, qMA)): problème de
## convergence possible : optim renvoie un code = 1
```

```
res_choice
```

```
##          Pvalue      Decision
## AR1      0.001 NO WHITE NOISE
## nonparam 0.944  WHITE NOISE
## ARMA 1 1  0.003 NO WHITE NOISE
```

```
print(res_choice$Pvalue)
```

```
## [1] "0.001" "0.944" "0.003"
```

On observe que la p-valeur la plus élevée est obtenue avec le modèle de Toeplitz non paramétrique, suggérant qu'il capture mieux la structure de dépendance des résidus. On va donc utiliser cette méthode pour le blanchiment des résidus.

Vérifions la différence des deux p-valeurs entre le modèle AR(1) et le modèle Toeplitz non paramétrique.

```
# Whitening AR(1)
sqrt_inv_sigma_AR1 <- whitening(residus, "AR1", pAR = 1, qMA = 0)

# Whitening Toeplitz Non-paramétrique
sqrt_inv_sigma_nonparam <- whitening(residus, "nonparam", pAR = 1, qMA = 0)

# Résidus blanchis avec Toeplitz non-param
residus_nonparam <- residus %*% as.matrix(sqrt_inv_sigma_nonparam)

# Résidus blanchis AR(1)
residus_AR1 <- residus %*% as.matrix(sqrt_inv_sigma_AR1)

pvalue_non_param = whitening_test(residus_nonparam)
print(pvalue_non_param)
```

```
## [1] 0.9441191
```

```
pvalue_AR1 = whitening_test(residus_AR1)
print(pvalue_AR1)
```

```
## [1] 0.001275907
```

On voit que la p-valeur du modèle AR(1) est très faible, ce qui est très loin d'une p-valeur acceptable.

5) Sélection des gènes influencés par les facteurs expérimentaux par la Stability Selection

Maintenant que nous avons stabilisé les résidus via le blanchiment, nous pouvons procéder à la sélection des gènes influencés par le facteur dose. Nous allons utiliser la Stability Selection, une méthode robuste pour identifier les variables pertinentes dans un contexte de haute dimension. L'idée est de répéter plusieurs fois le processus de sélection sur des sous-échantillons des données et de calculer la fréquence de sélection de chaque gène apparu dans les couples variable-réponse.

Cette méthode peut-être très lourde car elle nécessite le calcul de la matrice de Kronecker. Ici...

```
# require(doMC)
# registerDoMC(cores = 3)
# Freqs = variable_selection(Y, X, sqrt_inv_sigma_nonparam, nb_repli = 200, parallel = TRUE, nb.cores = 3)
# save(Freqs, file = 'Freqs_liver_toxicity_TOEPLITZ_nbreplis_200.Rdata')
load('Freqs_liver_toxicity_TOEPLITZ_nbreplis_200.Rdata')
```

```
dim(X)
```

```
## [1] 64  4
```

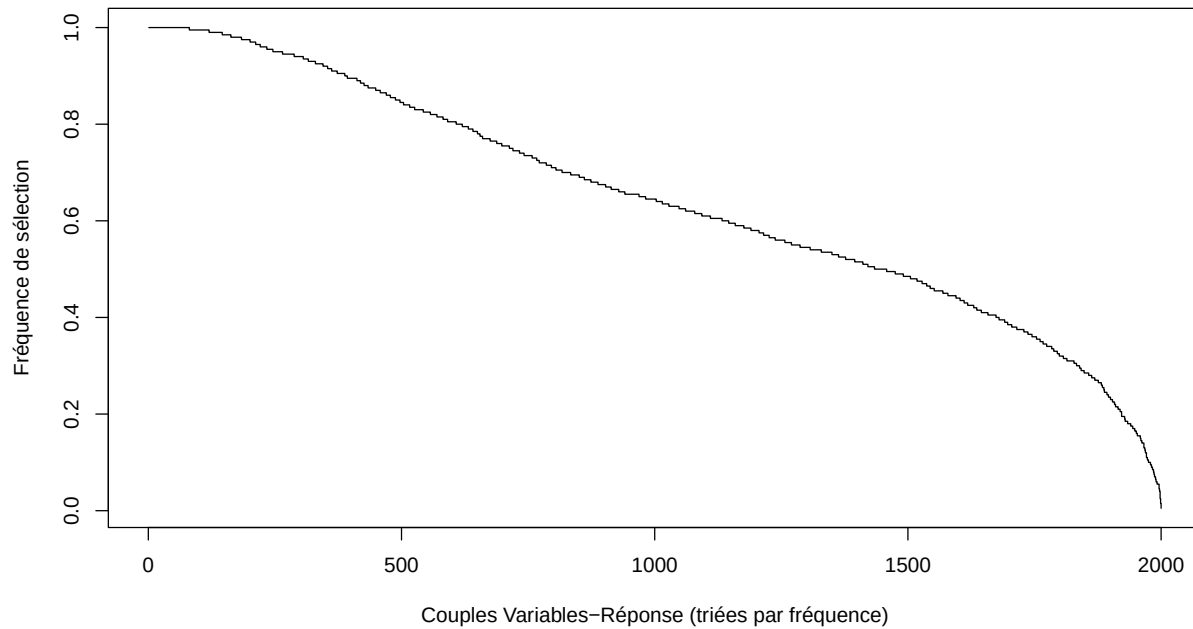
Renommons les colonnes de Freqs pour plus de clarté

```
colnames(Freqs) <- c('Names_of_Y', 'Names_of_X', 'frequency')
```

Visualisons la fréquence des couples variables-réponse

```
plot(sort(Freqs$frequency, decreasing = T), type = 's',
      main = "Fréquence de sélection des couples variables-réponse",
      xlab = "Couples Variables-Réponse (triées par fréquence)", ylab = "Fréquence de sélection")
```

Fréquence de sélection des couples variables-réponse



```
sort(Freqs$frequency, decreasing = T)[1:50]
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [39] 1 1 1 1 1 1 1 1 1 1 1 1
```

**

Pour sélectionner les gènes les plus influencés, vu le contexte de santé médicale, fixons un seuil de fréquence de sélection à 0.93, ce qui signifie que l'on retient les gènes sélectionnés dans au moins 93 % des répétitions.

```
# Fixer un seuil de fréquence de sélection
seuil <- 0.93
indices <- which(Freqs$frequency >= seuil)
length(indices)
```

```
## [1] 329
```

```
# Gènes sélectionnés
selected_genes <- unique(Freqs[indices, ]$Names_of_Y)
length(selected_genes)
```

```
## [1] 193
```

```
# Vérifier combien de couples liés à la dose sont retenus
sum(grepl("Dose", Freqs$Names_of_X) & (1:nrow(Freqs) %in% indices))
```

```
## [1] 329
```

193 gènes sont sélectionnés avec ce seuil.

E) Visualisation et interprétation des résultats

Pour avoir des visuels lisibles, on va regarder les 15 gènes les plus influencés. Nous avons besoin de réestimer les coefficients pour ces gènes car le LASSO introduit un biais.

Reestimation des coefficients pour les variables sélectionnés

```
# # Étape 1 : construction des vecteurs Yvec et Xvec
# Yvec <- as.numeric(Y %%% sqrt_inv_sigma_nonparam)

# Xvec <- kronecker(t(sqrt_inv_sigma_nonparam), X)

# # Étape 2 : sélection des colonnes correspondant aux variables retenues
# Xvec_sel <- Xvec[, indices]

# # Étape 3 : estimation des coefficients (solution des moindres carrés)
# B_sel_hat <- solve(t(Xvec_sel) %%% Xvec_sel, t(Xvec_sel) %%% Yvec)

# # Étape 4 : stockage dans Freqs
# p <- ncol(X)
# q <- ncol(Y)
# Freqs$estim <- rep(0, p * q)
# Freqs$estim[indices] <- as.vector(B_sel_hat)
# # Save Freqs with estim
# save(Freqs, file = 'Freqs_liver_toxicity_TOEPLITZ_nbreplic_200_with_estim.Rdata')
load('Freqs_liver_toxicity_TOEPLITZ_nbreplic_200_with_estim.Rdata')
```

Sélection des 15 gènes les plus influencés par les valeurs réestimés.

```
library(dplyr)

##
## Attachement du package : 'dplyr'

## Les objets suivants sont masqués depuis 'package:stats':
##
##     filter, lag

## Les objets suivants sont masqués depuis 'package:base':
##
##     intersect, setdiff, setequal, union
```

```

# --- Sélection des 15 plus influents POUR DOSE
Freqs_dose <- Freqs %>%
  filter(grepl("Dose", Names_of_X), frequency >= seuil)

top15_dose <- Freqs_dose %>%
  arrange(desc(abs(estim))) %>%
  head(15)

top15_dose$Effet <- top15_dose$Names_of_X
top15_dose

```

##	Names_of_Y	Names_of_X	frequency	estim	Effet
## 1	A_43_P20967	Dose.Group2000	1.000	-1.2935503	Dose.Group2000
## 2	A_42_P537398	Dose.Group2000	1.000	-1.2915065	Dose.Group2000
## 3	A_42_P730438	Dose.Group2000	1.000	-1.1640317	Dose.Group2000
## 4	A_43_P14795	Dose.Group2000	1.000	-1.1396446	Dose.Group2000
## 5	A_42_P815827	Dose.Group1500	1.000	-1.1046912	Dose.Group1500
## 6	A_42_P672425	Dose.Group2000	0.995	-1.0884747	Dose.Group2000
## 7	A_43_P14952	Dose.Group2000	1.000	-1.0876878	Dose.Group2000
## 8	A_42_P474743	Dose.Group2000	1.000	-1.0830160	Dose.Group2000
## 9	A_42_P493879	Dose.Group50	1.000	-1.0426540	Dose.Group50
## 10	A_43_P17074	Dose.Group2000	0.995	1.0182900	Dose.Group2000
## 11	A_42_P669609	Dose.Group50	1.000	1.0171184	Dose.Group50
## 12	A_42_P565061	Dose.Group2000	0.985	-1.0095312	Dose.Group2000
## 13	A_42_P755887	Dose.Group2000	1.000	-0.9968025	Dose.Group2000
## 14	A_42_P772157	Dose.Group50	1.000	-0.9966881	Dose.Group50
## 15	A_42_P839593	Dose.Group2000	1.000	-0.9940519	Dose.Group2000

Heatmap des effets des facteurs sur 10 gènes parmi les gènes sélectionnés

```

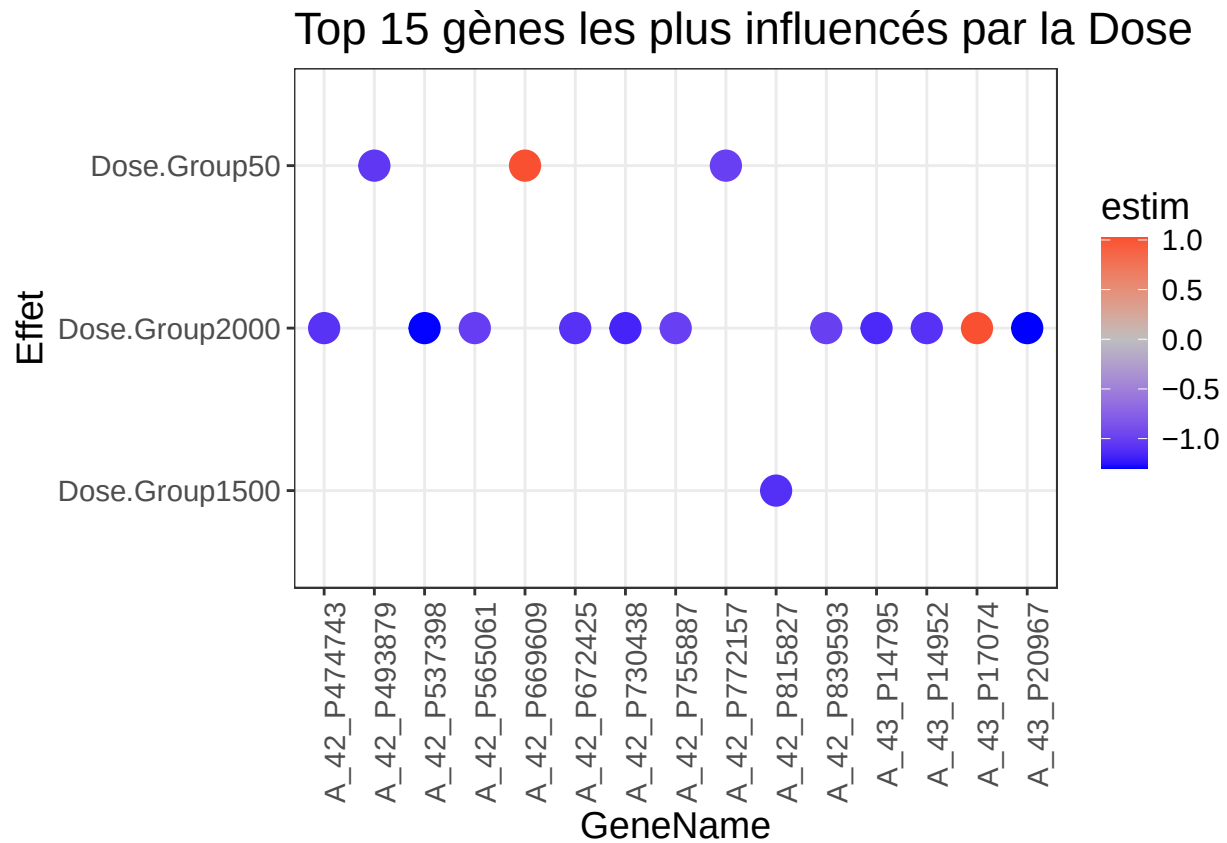
library(ggplot2)
library(dplyr)

# Récupérer les noms lisibles des gènes depuis table4
top15_dose <- top15_dose %>%
  left_join(table4, by = c("Names_of_Y" = "geneBank")) %>%
  mutate(GeneName = ifelse(!is.na(gene.title), gene.title, Names_of_Y))

# Heatmap : gènes (axe X) vs effets Dose (axe Y)
gr <- ggplot(top15_dose, aes(
  x = GeneName,
  y = Effet,
  color = estim
)) +
  geom_point(size = 5) +
  scale_color_gradient2(low = "blue", mid = "grey", high = "red") +
  theme_bw() +
  theme(
    axis.text.x = element_text(angle=90, hjust=1),
    text = element_text(size=14)
  )

```

```
) +
  ggtitle("Top 15 gènes les plus influencés par la Dose")
print(gr)
```



```
# Sauvegarde
ggsave("heatmap_top15_dose.png", plot = gr, width = 12, height = 8)
```

On observe que certains gènes sont fortement influencés par le facteur dose

Boxplots de 10 gènes parmi les gènes sélectionnés

```
library(reshape2)
library(ggplot2)
library(dplyr)

# Gènes à tracer
genes_to_plot <- top15_dose$Names_of_Y

# Construction du tableau réduit
table_red <- as.data.frame(Y[, genes_to_plot, drop = FALSE])
```

```

# Ajout de la colonne Dose
table_red$Dose <- factor(table_XY$Dose.Group)

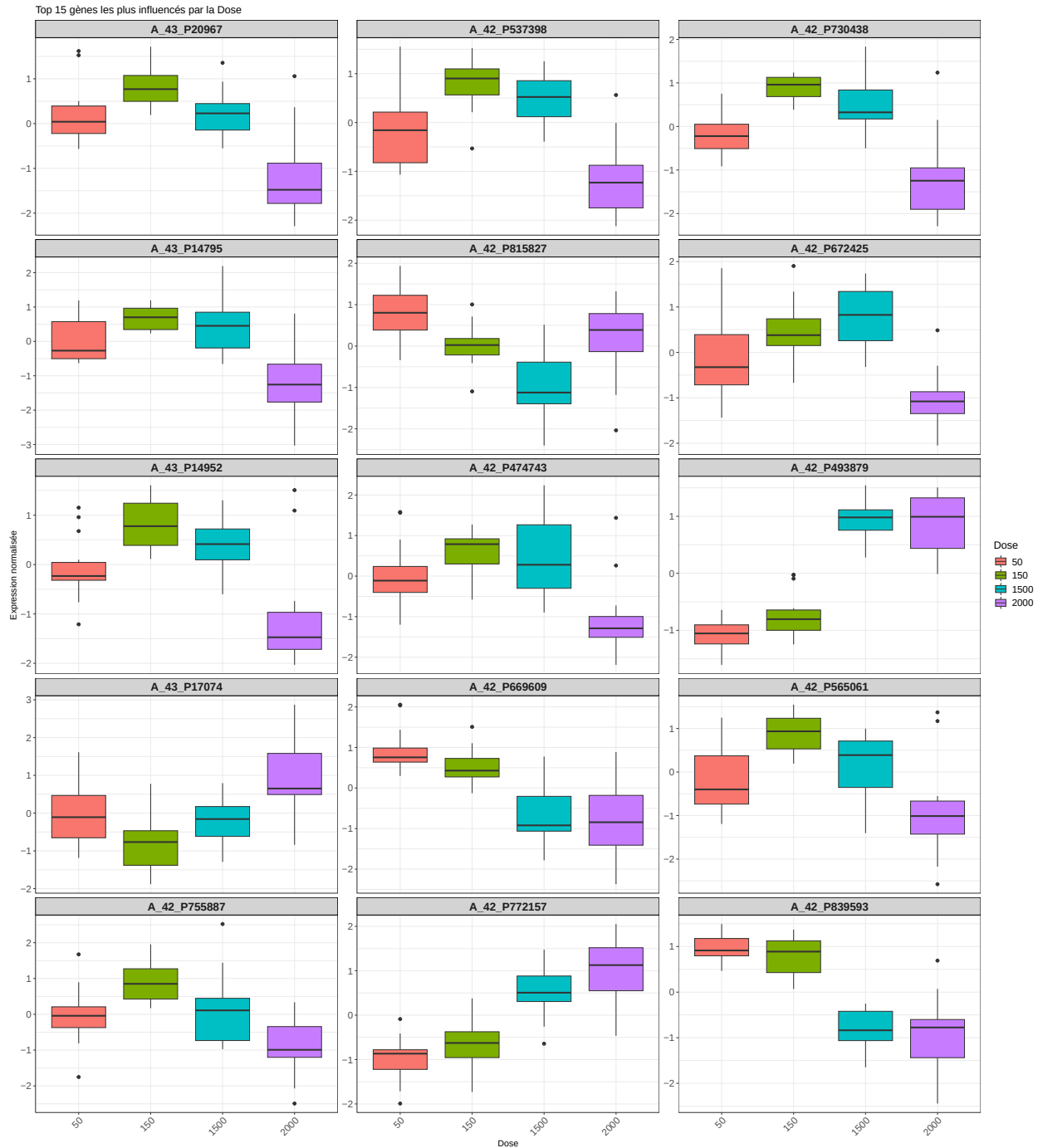
# Passage au format long
table_long <- melt(table_red, id.vars = "Dose")

# Forcer les noms de gènes comme facteur pour affichage correct
table_long$GeneLabel <- factor(table_long$variable, levels = genes_to_plot)

# Création du graphique
g <- ggplot(table_long, aes(x = Dose, y = value, fill = Dose)) +
  geom_boxplot() +
  facet_wrap(~GeneLabel, scales = "free_y", ncol = 3) +
  theme_bw() +
  theme(
    axis.text.x = element_text(size=12, angle=45, hjust=1),
    axis.text.y = element_text(size=12),
    strip.text = element_text(size=14, face="bold"),
    strip.background = element_rect(fill = "lightgrey", color = "black"),
    legend.position = "right",
    legend.title = element_text(size=13),
    legend.text = element_text(size=12)
  ) +
  ylab("Expression normalisée") +
  xlab("Dose") +
  ggtitle("Top 15 gènes les plus influencés par la Dose")

print(g)

```



```
# Sauvegarde
ggsave("boxplot_top15_dose.png", plot = g, width = 16, height = 12)
```

On voit que les gènes sélectionnés montrent des variations d'expression significatives en fonction de la dose.

Dendrogramme de 10 gènes parmi les gènes sélectionnés

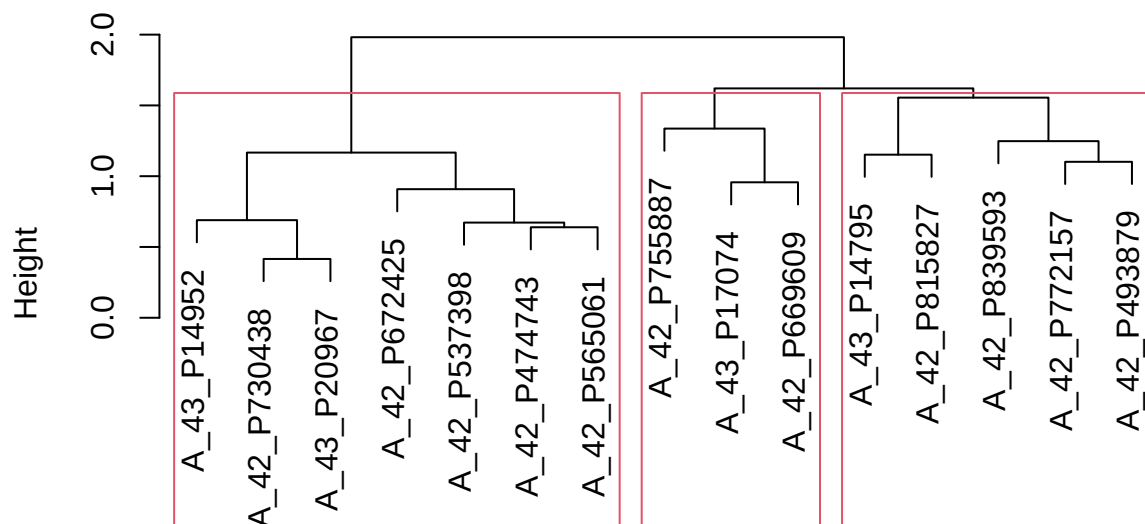
```
# Sélection des gènes
liste_genes_selected <- unique(top15_dose$Names_of_Y)
length(liste_genes_selected)

## [1] 15

# Extraction des résidus pour ces gènes
residus_red <- residus[, colnames(residus) %in% liste_genes_selected]

# Construction du dendrogramme
hc <- hclust(dist(1 - abs(cor(residus_red))))
plot(hc, main = "Dendrogramme des gènes influencés par la Dose")
rect.hclust(hc, k = 3)
```

Dendrogramme des gènes influencés par la Dose



$\text{dist}(1 - \text{abs}(\text{cor}(\text{residus_red})))$
 $\text{hclust}(*, \text{"complete"})$

```
# Sauvegarde du graphique
dev.copy(png, "Dendrogram_top15_genes_dose.png", width = 800, height = 600)
```

```
## png
## 3
```

```
dev.off()
```

```
## pdf  
## 2
```

On observe des clusters de gènes avec des profils d’expression similaires parmi les gènes sélectionnés, ce qui peut indiquer des régulations transcriptionnelles communes en réponse à la dose. Cela suggère que ces gènes pourraient être impliqués dans des voies biologiques similaires affectées par le paracétamol.

F) Difficultés rencontrées

La matrice de Design X a posé beaucoup de confusion à cause de l’oubli d’utilisation de `factor` pour le facteur Dose.Group, ce qui a conduit à des dimensions incorrectes. Modéliser X avec 2 facteurs (Dose et Temps) dès le début a provoqué cette confusion avec des mauvaises dimensions, on a décidé pour le moment de ne modéliser que l’effet principal de la Dose.

La deuxième grande difficulté a été la lourdeur computationnelle de la Stability Selection, en particulier le calcul de la matrice de Kronecker, gènes. Pour alléger les calculs, nous avons réduit la dimensionnalité en sélectionnant les 500 gènes les plus variables.

Le choix du modèle de dépendance des résidus a également été crucial pour obtenir des résidus blanchis satisfaisants. Dans un premier temps, à cause d’une autre approche de modélisation, nous avons eu des p-valeurs de blanchiments très faibles pour les 3 familles de modèles, ce qui nous a permis d’explorer la modélisation par les matrices de covariances par blocs.

NB: Pour la fonction `whitening_choice` de `MultiVarSel`, les p-valeurs affichées peuvent s’afficher ZERO alors qu’elles sont supérieures au seuil enregistrable de R, c’est à dire inférieur à environ 10^{-301}

G) Conclusion

Ce projet fut très formateur et a permis d’appliquer des techniques avancées de modélisation statistique multivariée à un problème réel de toxicité médicamenteuse.

La mise en situation dans un environnement complexe par la contrainte des ressources disponibles a été un défi intéressant à relever.